Machine Learning Fundamentals

1. Introduction to Machine Learning

What is Machine Learning?

Machine Learning (ML) is a branch of Artificial Intelligence (AI) that enables computers to learn patterns from data and make predictions or decisions without being explicitly programmed with every rule.

Example: Instead of writing a fixed rule to predict a student's exam score, we provide examples of study hours and scores. An ML model learns the relationship and predicts scores for new students.

Traditional Programming vs. Machine Learning

Traditional Programming	Machine Learning
Rules + Data → Output	Data + Expected Outputs → Learned Model
A programmer writes the rules.	The algorithm learns patterns from examples.
Best when rules are known and stable.	Useful when patterns are complex or difficult to specify manually.

Basic ML Terminology

- **Dataset:** A collection of examples used for learning or evaluation.
- **Sample/instance:** One example in a dataset.
- **Feature:** An input variable used to make a prediction.
- **Label/target:** The value the model is expected to predict.
- **Model:** A learned mathematical function that maps inputs to predictions.
- **Training:** The process of learning model parameters from data.
- **Prediction:** The output produced by a trained model.
- **Generalization:** How well a model performs on unseen data.

A model can be represented as:

$$\hat{y} = f_\theta(X)$$

where:

- X = input features
- f = model/function
- θ = learned parameters
- $\hat{y}$ = predicted output

The goal is not simply to memorize training data. A useful model should **generalize** to new examples.

2. Linear Regression: Mathematical Foundation

What is Linear Regression?

Linear Regression is a supervised learning algorithm used to predict a **continuous numerical value**, such as salary, house price, temperature, or exam score.

It models a linear relationship between input x and predicted output ŷ:

$$\hat{y} = wx + b$$

where:

- x = input feature
- w = weight or slope
- b = bias or intercept
- ŷ = predicted value

Geometric Intuition

Each training example is represented as a point on a graph. Linear Regression finds a line that fits the data as well as possible, typically by minimizing the sum of squared prediction errors.

- **Slope w:** Indicates how much the prediction changes when x increases by one unit.
- **Intercept b:** The predicted value when x = 0.

Example

Suppose a model predicts salary from years of experience:

Salary = 5000 × Experience + 20000

For 3 years of experience:

ŷ = 5000(3) + 20000 = 35000

The predicted salary is 35,000 in the salary's stated units.

Other Common Model Types

- **Polynomial Regression:** Models nonlinear relationships by including powers of input features, such as x^2.
- **Logistic Regression:** Commonly used for classification. It can estimate a probability using the sigmoid function: **σ(z) = 1 / (1 + e^(−z))**, where z = wx + b.
- **Decision Tree:** Makes predictions using a sequence of feature-based rules.
- **K-Nearest Neighbors (KNN):** Predicts using nearby training examples.
- **Neural Network:** Learns patterns through interconnected computational units and layers.

3. Types of Machine Learning

3.1 Supervised Learning

The model learns from **labeled data**, where each input has a known target.

$$\mathbf{D = \{(x_i, y_i)\}, i = 1…n}$$

Here, x_i is an input and y_i is its known target.

Main Tasks

Regression: Predicts a continuous value.

Examples:

- House-price prediction
- Temperature forecasting
- Salary estimation

Common algorithms:

- Linear Regression
- Lasso Regression
- Polynomial Regression
- SVM Regressor
- Decision Tree Regression
- Random Forest Regression
- Bayesian Linear Regressor

Classification: Predicts a category or class.

Examples:

- Spam vs. non-spam email
- Disease-positive vs. disease-negative
- Image classification

Common algorithms:

- Logistic Regression
- KNN
- Decision Trees
- Support Vector Machines (SVMs)
- Random Forest
- Naïve Bayes Classifier

3.2 Unsupervised Learning

The model receives data **without target labels** and attempts to discover patterns or structure.

$$\mathbf{D = \{x_i\}, i = 1…n}$$

Common tasks:

- **Clustering:** Groups similar examples, such as customer segmentation.
- **Dimensionality reduction:** Represents data using fewer variables while retaining useful information.
- **Association-rule learning:** Finds items or events that frequently occur together, such as products often bought in the same transaction.

K-means is a popular clustering algorithm. It assigns examples to K clusters and updates their centers to reduce the within-cluster squared distances.

Common algorithms:

- K-Means Clustering
- Hierarchical Clustering
- Principal Clustering
- Apriori
- Eclat

For association rules:

- **Support** measures how frequently an itemset appears.
- **Confidence** measures how often the rule's consequent occurs when its antecedent occurs.

Applications include customer segmentation, data exploration, and recommendation systems.

3.3 Semi-Supervised Learning

Uses a small amount of labeled data together with a larger amount of unlabeled data. It is useful when labeling examples is expensive or time-consuming.

3.4 Reinforcement Learning

An **agent** interacts with an **environment**, takes actions, and receives rewards or penalties. It learns a policy that aims to maximize cumulative reward.

Examples include game-playing agents and robot control.

4. Model Selection and Cross-Validation

What is Model Selection?

Model selection means choosing a suitable model and its settings for a particular task and dataset.

- The type of problem:
    - Regression Task – Linear Regression , Random Forest , Polynomial Regression
    - Classification Task – SVM , Logistic Regression, Decision trees
    - Clustering Task – K-Means Clustering , Hierarchical Clustering

- The size and characteristics of the dataset
    - Images & Videos – CNN
    - Text or Speech data – RNN
    - Numerical data – SVM, Logistic Regression, Decision trees etc
- Predictive performance on validation data.

What is Cross-Validation?

Cross-validation estimates how well a model may perform on unseen data by repeating training and validation across different data splits.

5-Fold Cross-Validation

1. Divide the training data into five approximately equal folds.
2. Train on four folds and validate on the remaining fold.
3. Repeat until each fold has been used for validation once.
4. Calculate the average of the five validation scores.

Mean Score = (1/5) × Σ $Score_i$, for i = 1…5

Cross-validation helps compare models more reliably than relying on one validation split alone.

Important: Keep a separate test set for final evaluation. For time-series data, use time-aware splits rather than randomly mixing past and future observations. Grouped data may also require group-aware splitting to prevent leakage.

5. Overfitting

Definition

Overfitting occurs when a model learns the training data too closely, including noise or accidental patterns, and performs poorly on unseen data.



Typical pattern:

- Training performance: very good
- Validation/test performance: noticeably worse

Causes

- Model is too complex for the available data.
- Training data is limited or unrepresentative.
- Data contains noise.
- The model is trained for too long in some settings.

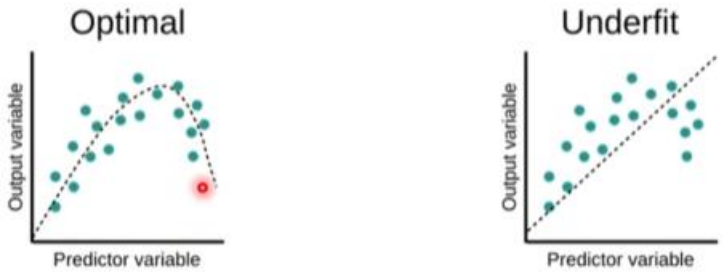
How to Reduce Overfitting

- Collect more representative training data when possible.
- Reduce model complexity.
- Apply **regularization**, which penalizes overly complex parameter values.
- Use cross-validation for model selection.
- Use early stopping when validation performance stops improving.
- Use dropout in suitable neural-network architectures.
- Check for data leakage and duplicate or overlapping examples across splits.

6. Underfitting

Definition

Underfitting occurs when a model is too simple or has not learned enough to capture the important patterns in the data.



Typical pattern:

- Training performance: poor
- Validation/test performance: also poor

Causes

- Model is too simple / Wrong model choose
- Important features are missing , less complexity of the model .
- The model has not been trained sufficiently.
- Regularization is too strong.
- Less variance but high bias.

How to Reduce Underfitting

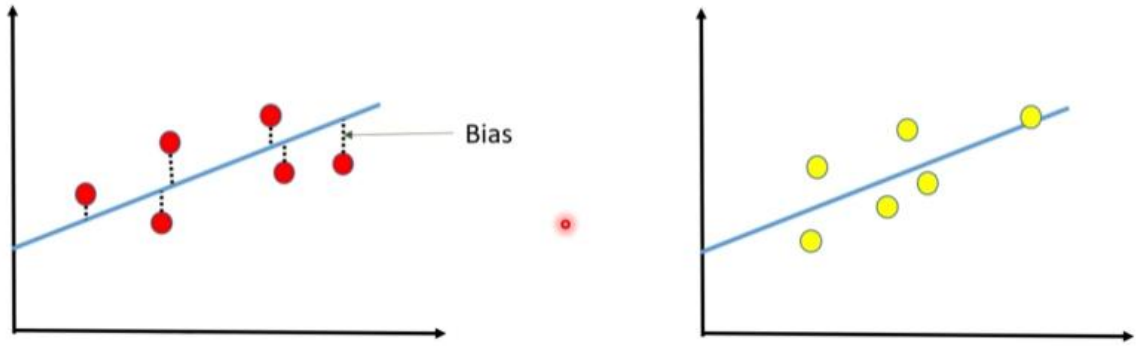
- Use a more expressive model.
- Add useful features or improve feature representation.
- Train for longer when insufficient training is the cause.
- Reduce excessive regularization.
- Check that the learning process and data preprocessing are correct.

7. Bias–Variance Tradeoff

The bias–variance tradeoff describes two sources of prediction error.

Bias

Bias is an error caused by simplifying assumptions that prevent the model from capturing the true relationship. It is the difference between the average prediction of our model and the correct value which we are trying to predict.



- High bias is often associated with underfitting.
- Example: using a straight line to model a strongly curved relationship.

Variance

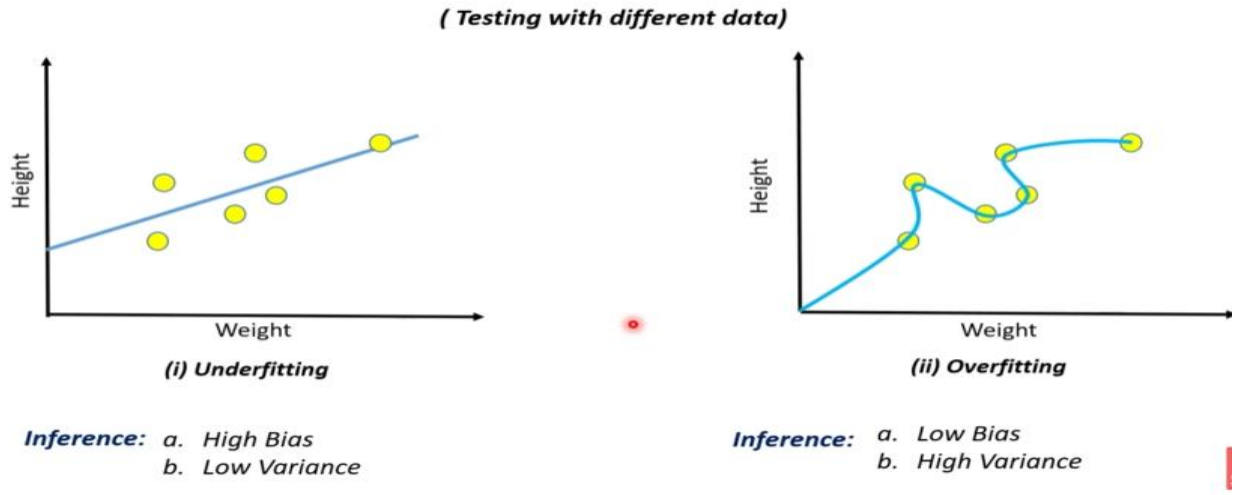
Variance describes how much a model's predictions change when it is trained on different samples of the data.

- High variance is often associated with overfitting.
- Example: a very complex model that changes substantially when a few training examples change.

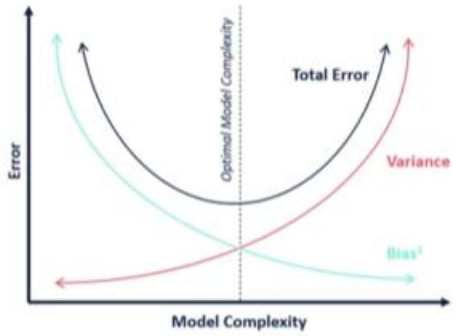
A common conceptual decomposition of expected prediction error is:

Expected Error ≈ Bias² + Variance + Irreducible Noise

The exact decomposition depends on the assumptions and loss being considered.



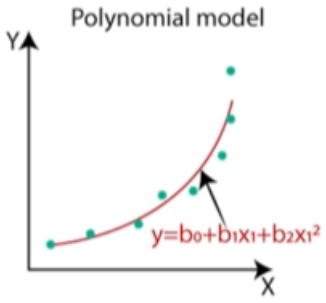
The Tradeoff



- A model that is too simple may have high bias.
- A model that is too complex may have high variance.
- The aim is to find a suitable balance that performs well on unseen data.

Techniques to have better Bias – Variance Tradeoff:

- Good model selection
- Regularization
- Dimensionality Reducion
- Ensembel methods



8. Loss Functions

What is a Loss Function?

A loss function measures the error between a model's prediction

and the true target. During training, an optimization algorithm adjusts the model parameters to reduce the loss. It is helpful to determine which model performs better & which parameters(co-efficients) are better.

For one example, loss can be written as:

$$L(y, \hat{y}) = (1/n) \times \Sigma (y_i - \hat{y}_i)^2$$

where y is the true value and ŷ is the prediction. Low loss value means High Accuracy.

Mean Squared Error (MSE)

A common loss for regression is Mean Squared Error:

$$MSE = (1/n) \times \Sigma (y_i - \hat{y}_i)^2, \text{ for } i = 1 \ldots n$$

It averages the squared differences between actual and predicted values. Squaring makes larger errors contribute more strongly.

Example: If actual values are [3, 5] and predictions are [2, 7], the squared errors are 1 and 4, so:

MSE = (1 + 4) / 2 = 2.5

Other Loss Functions

- **Mean Absolute Error (MAE):** Average absolute prediction error; less sensitive to large errors than MSE.
- **Binary Cross-Entropy:** Commonly used for binary classification.
- **Categorical Cross-Entropy:** Commonly used for multiclass classification.
- **Kullback–Leibler (KL) Divergence:** Measures a difference between probability distributions; it is often used in probabilistic learning.

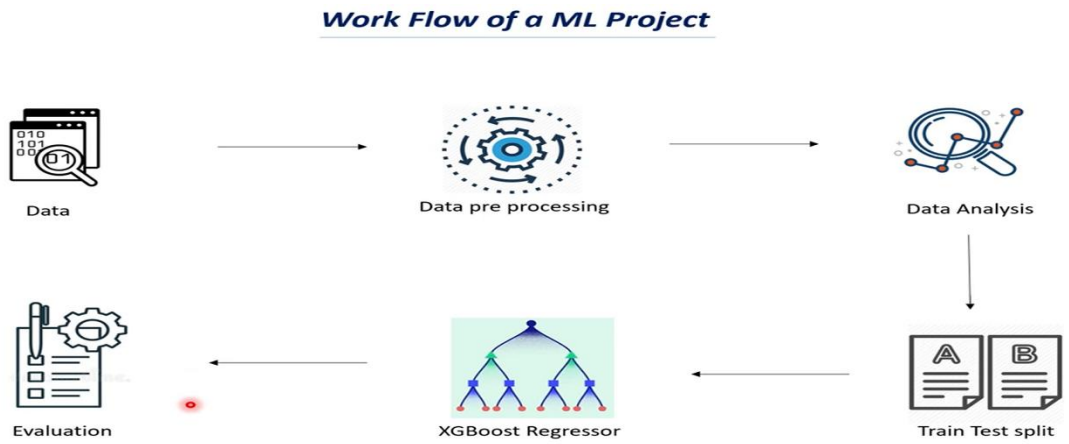
Loss vs. Evaluation Metric

A **loss function** is the quantity optimized during training. An **evaluation metric** is used to assess performance for a task. They may be the same, but they do not have to be.

9. Model Evaluation

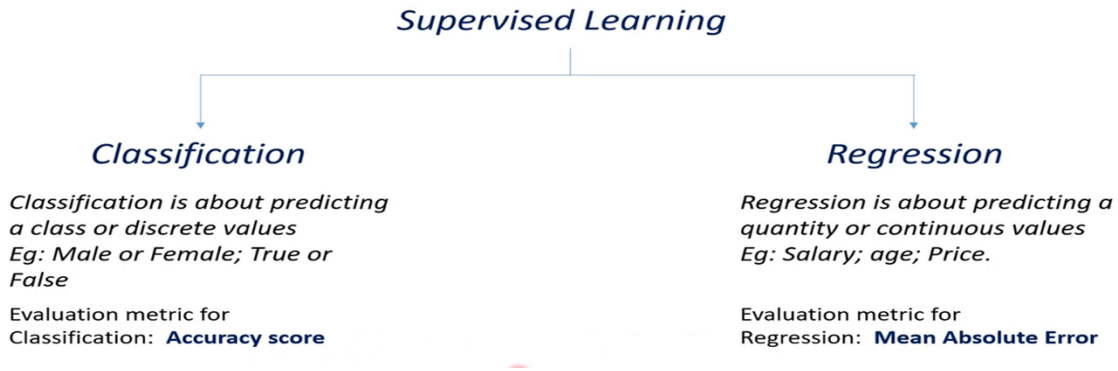
A model must be evaluated on data that was not used to fit its parameters. A common workflow is:

1. Split data into training, validation, and test sets, as appropriate.
2. Fit preprocessing steps and the model using training data only.
3. Use validation data for model selection and tuning.
4. Evaluate the selected model on the test set once final decisions are made.



Never use the test set repeatedly to tune the model, because this can make the test score overly optimistic.

We know,



9.1 Classification Metrics

Accuracy is the proportion of predictions that are correct.

Accuracy = Correct Predictions / Total Predictions

If 128 out of 150 predictions are correct:

Accuracy = 128 / 150 × 100 ≈ 85.33%

Accuracy can be misleading when classes are imbalanced. In python, we don't have to do it manually. We can use the library function: **from sklearn.metrics import accuracy_score**

Other useful metrics:

- **Precision:** Of all examples predicted positive, the proportion that are truly positive. **Precision = TP / (TP + FP)**
- **Recall:** Of all truly positive examples, the proportion correctly identified. **Recall = TP / (TP + FN)**
- **F1-score:** Harmonic mean of precision and recall. **F1 = 2 × (Precision × Recall) / (Precision + Recall)**

Here, TP = true positives, FP = false positives, and FN = false negatives.

9.2 Regression Metrics

- **MSE:** Average squared error.
- **MAE:** Average absolute error.
- **R^2 score:** Measures how much of the target's variation is accounted for by the model relative to a mean-prediction baseline. It can be negative on test data.

Choose metrics according to the task and the cost of different errors. For example, recall may be especially important when missing a positive medical case is costly.

In Python, common metrics are available in `sklearn.metrics`

10. Model Parameters and Hyperparameters

Model Parameters

Parameters are values learned from training data. These can be considered as internal parameter.

Examples:

- Weights and bias in Linear Regression
- Weights in a neural network
- Split thresholds learned by a Decision Tree

For Linear Regression: $\hat{y} = wx + b$. The training process learns w and b.

Here,

w = weights; which decides how much influence the input will have on the output. (Desirable features will have the positive weight values)

b = bias; it is the offset value. Used to shift the model in a particular direction.

Applicant's Details

Name	Degree	College	C	C++	Python	Height	Weight	No. of Backlogs
A	B.E	ABC college	✓	✗	✓	165	72	1
B	M.E	XYZ College	✓	✓	✗	168	80	0
C	M.C.A	State College	✓	✗	✗	175	67	0
D	B.E	ZYX College	✓	✓	✓	168	70	2
✗	✓	✓	✓	✓	✓	✗	✗	✓

Hyperparameters

Hyperparameters are parameters whose values control the learning process. These are adjustable parameters used to obtain an optimal model. The settings are chosen before or during model selection.

They are not usually learned directly as ordinary model parameters from the training objective that's why they are external parameters.

Examples:

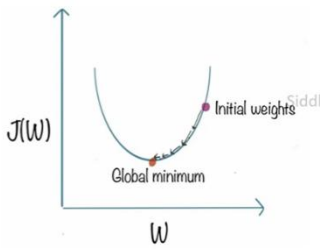
- Learning rate
- Number of training epochs
- Batch size
- Maximum depth of a Decision Tree
- Number of neighbors K in KNN
- Regularization strength

Hyperparameters are commonly selected using validation data or cross-validation.

Parameters	Hyperparameters
Learned from training data	Set or selected by the practitioner or tuning procedure
Example: weight w, bias b	Example: learning rate, tree depth
Directly define the fitted model	Control the model structure or training process

Note: here,
Learning Rate - is a tuning parameter in an optimization algorithm. Determines the step size at each iteration while moving toward a minimum of a loss function.

Number of Epochs – represents the number of times the model iterates over the entire dataset. High number of epoch creates a chance of model overfitting.



Gradient Descent

11. Gradient Descent

For a model, optimization refers to determining the best parameters, such that the loss function decreases, as a result of which the model can predict more accurately.

What is Gradient Descent?

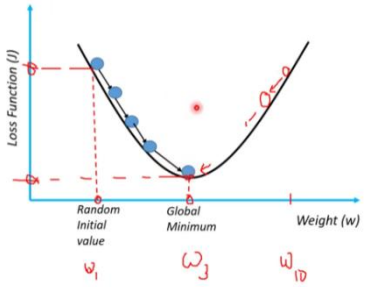
Gradient Descent is an optimization algorithm that updates model parameters in the direction that reduces the loss.

For a parameter θ:

$$\theta_{new} = \theta_{old} - \eta \times \partial L/\partial \theta$$

where:

- θ = a model parameter(weight/bias)
- L = loss function
- ∂L/∂θ = gradient of the loss with respect to that parameter
- η = learning rate



The gradient indicates the direction of steepest increase locally, so subtracting it moves the parameter toward lower loss.

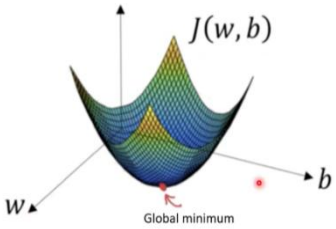
Gradient Descent in Linear Regression

For $\hat{y} = wx + b$, the parameters w and b are updated using their gradients:

$w_{new} = w_{old} - \eta \times \partial L/\partial w$

$b_{new} = b_{old} - \eta \times \partial L/\partial b$

The process is repeated to reduce the loss.



Learning Rate

The learning rate controls the size of each update.

- **Too small:** Training may be very slow.
- **Too large:** Updates may overshoot a minimum or fail to converge.
- **Suitable:** Training can make steady progress toward a low-loss solution.

Epoch

An **epoch** is one complete pass through the training dataset. Multiple epochs may be needed to learn useful patterns, but too many can contribute to overfitting in some models.

Types of Gradient Descent

- **Batch Gradient Descent:** Computes each update using the full training dataset.
- **Stochastic Gradient Descent (SGD):** Computes each update using one training example.
- **Mini-batch Gradient Descent:** Computes each update using a small batch of examples. This is widely used in neural-network training.

Gradient Descent does not guarantee finding a global minimum for every problem. Its behavior depends on the loss surface, initialization, learning rate, and other settings.

12. Common Questions and Answers

Q1. What is the main goal of Machine Learning? To learn patterns from data and make useful predictions or decisions on new examples.

Q2. What is the difference between regression and classification? Regression predicts numerical values, while classification predicts categories.

Q3. What is the difference between overfitting and underfitting? Overfitting means performing well on training data but poorly on unseen data. Underfitting means failing to learn the important patterns even in the training data.

Q4. Why do we use cross-validation? To estimate generalization performance more reliably and compare models using multiple training/validation splits.

Q5. What is the difference between a parameter and a hyperparameter? A parameter is learned from training data; a hyperparameter controls the model or its training process and is selected separately.

Q6. Why is a loss function needed? It quantifies prediction error and provides an objective for model optimization.

Q7. Why is accuracy not always sufficient? With imbalanced classes, a model can achieve high accuracy while missing many examples from the minority class. Precision, recall, F1-score, or other task-specific metrics may be more informative.

Q8. What does the learning rate do? It controls the size of parameter updates during optimization.

Q9. What is generalization? The ability of a trained model to perform well on data it has not seen during training.

Q10. Why should the test set be kept separate? It provides a final, less biased estimate of performance on unseen data. Using it repeatedly for tuning can cause information leakage into model selection.

13. Quick Revision Sheet

Topic	Key Point
Machine Learning	Learns patterns from data
Feature	Input used to make a prediction
Label/target	Value to be predicted
Supervised learning	Uses labeled data
Unsupervised learning	Finds structure in unlabeled data
Regression	Predicts a continuous value
Classification	Predicts a class or category
Linear Regression	$\hat{y} = wx + b$
Overfitting	Good training performance, weaker unseen-data performance
Underfitting	Poor performance on both training and unseen data
Bias	Error from overly restrictive assumptions
Variance	Sensitivity to changes in training data
Loss function	Quantifies prediction error during training
Cross-validation	Repeated training/validation across data folds
Parameter	Learned value, such as a weight
Hyperparameter	Setting, such as learning rate or tree depth
Gradient Descent	Updates parameters to reduce loss
Epoch	One complete pass through training data
Accuracy	Fraction of predictions that are correct
Precision	Fraction of predicted positives that are truly positive
Recall	Fraction of actual positives correctly identified
Generalization	Performance on unseen data